

Step 4 — A Router That Sends Each Message to the Right Agent Automatically — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — keyword-based router

```
# agents/router.py
TRIGGER_WORDS = ["charged", "broken", "refund", "cancel", "not working", "can't l

def decide(message):
    lowered = message.lower()
    if any(word in lowered for word in TRIGGER_WORDS):
        return "triage"
    return "concierge"
```

```
# main.py
from config import load_config
from chat_client import create_client
from agents.router import decide
from agents.concierge_agent import concierge_agent
from agents.triage_agent import triage_agent

config = load_config()
client = create_client(config.openai_api_key)

history = []
while True:
    message = input("You: ")
    decision = decide(message)
    if decision == "triage":
        ticket = triage_agent(client, message)
        print(ticket)
    else:
        history = concierge_agent(client, history, message)
```

This works for clearly-worded messages. It's missing logging of each routing decision, and it will misroute anything that doesn't happen to contain one of your trigger words — like the README's "wondering about your refund policy, no rush" example, which is really a Concierge-style question, not a Triage complaint, but contains "refund."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — keyword-based router, logged

```
# agents/router.py
import logging
from typing import Literal

logger = logging.getLogger(__name__)

TRIGGER_WORDS = ["charged", "broken", "refund", "cancel", "not working", "can't l

def decide(message: str) -> Literal["concierge", "triage"]:
    lowered = message.lower()
    if any(word in lowered for word in TRIGGER_WORDS):
        decision: Literal["concierge", "triage"] = "triage"
    else:
        decision = "concierge"
    logger.info("routed message to %s: %r", decision, message)
    return decision
```

```
# agents/concierge_agent.py
from openai import OpenAI
from chat_client import stream_message

CONCIERGE_SYSTEM_PROMPT = {
    "role": "system",
    "content": "You are a friendly support concierge. Chat naturally and help wit
}

def concierge_agent(client: OpenAI, history: list[dict], message: str) -> list[di
    if not history:
        history.append(CONCIERGE_SYSTEM_PROMPT)
    history.append({"role": "user", "content": message})
    return stream_message(client, history)
```

```
# agents/triage_agent.py
from openai import OpenAI
from chat_client import triage_extract
from schemas import SupportTicket

def triage_agent(client: OpenAI, message: str) -> SupportTicket:
    return triage_extract(client, message)
```

```
# main.py
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, check_api_key_at_startup
from agents.router import decide
from agents.concierge_agent import concierge_agent
from agents.triage_agent import triage_agent

config = load_config()
logger = get_logger(__name__)
client = create_client(config)
check_api_key_at_startup(client)

history: list[dict] = []
print("Type 'quit' to exit.")

while True:
    message = input("You: ")
    if message.strip().lower() == "quit":
        break

    decision = decide(message)
    if decision == "triage":
        ticket = triage_agent(client, message)
        print("Ticket:", ticket)
    else:
        print("Assistant: ", end="", flush=True)
        history = concierge_agent(client, history, message)
```

Approach 2 — a small model call instead of keywords

```
def decide(client: OpenAI, message: str) -> Literal["concierge", "triage"]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
```

```

        {
            "role": "system",
            "content": "Reply with exactly one word: 'triage' if the message
                        'problem needing customer support, otherwise 'concier
        },
        {"role": "user", "content": message},
    ],
    max_tokens=5,
)
raw = response.choices[0].message.content.strip().lower()
decision: Literal["concierge", "triage"] = "triage" if "triage" in raw else "
logger.info("routed message to %s: %r", decision, message)
return decision

```

Difference from Basic: every routing decision is logged with the exact message and the outcome, which is the checklist's explicit requirement, not just a nice-to-have. Each agent has its own system prompt (Concierge's is only set once, the first time `history` is empty) so the two really do have separate personalities, not a shared one with an if-statement bolted on. `main.py` checks the API key once at startup before the loop even begins. Approach 2's model-call router handles wording the keyword list didn't anticipate — but both approaches here still only ever return "concierge" or "triage," even on a message that's genuinely ambiguous, like the README's refund-policy example.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a third "unclear" path, with a logged reason

```
from typing import Literal

RouterDecision = Literal["concierge", "triage", "unclear"]

def decide_with_confidence(client: OpenAI, message: str) -> tuple[RouterDecision, str]:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "Decide how to route this message. Reply with exactly one line: "
                    "'concierge', 'triage', or 'unclear' followed by a short reason. "
                    "like: triage - customer says they were charged twice. "
                    "Use 'unclear' only if you genuinely cannot tell which one fits."
                ),
            },
            {"role": "user", "content": message},
        ],
        max_tokens=30,
    )
    raw = response.choices[0].message.content.strip()
    decision_word, _, reason = raw.partition(" - ")
    decision_word = decision_word.strip().lower()
    if decision_word not in ("concierge", "triage", "unclear"):
        decision_word = "unclear"
    logger.info("routed message to %s (%s): %r", decision_word, reason.strip(), message)
    return decision_word, reason.strip()
```

```
# main.py
decision, reason = decide_with_confidence(client, message)
if decision == "triage":
    ticket = triage_agent(client, message)
    print("Ticket:", ticket)
elif decision == "concierge":
```

```

    history = concierge_agent(client, history, message)
else:
    print("Not sure I understood that as a chat message or a support issue —")
    print("could you say a bit more about what you need?")

```

Expected behavior: clear messages still route exactly like Intermediate. The README's ambiguous "refund policy, no rush" message now has a genuine third outcome available instead of being forced into "triage" just because it contains the word "refund" — check your logs to see what the model actually decided and why.

Approach 2 — a numeric confidence score instead of a reason string

```

def decide_with_confidence(client: OpenAI, message: str) -> tuple[RouterDecision,
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {
                "role": "system",
                "content": (
                    "Reply with exactly two things separated by a comma: "
                    "'concierge' or 'triage', then a confidence percentage from 0
                    "Example: triage, 85"
                ),
            },
            {"role": "user", "content": message},
        ],
        max_tokens=10,
    )
    raw = response.choices[0].message.content.strip().lower()
    decision_word, _, confidence_str = raw.partition(",")
    decision_word = decision_word.strip()
    try:
        confidence = int(confidence_str.strip().rstrip("%"))
    except ValueError:
        confidence = 0

    if decision_word not in ("concierge", "triage") or confidence < 60:
        decision: RouterDecision = "unclear"
    else:
        decision = decision_word

    logger.info("routed message to %s (confidence=%d): %r", decision, confidence,
    return decision, confidence

```

Difference between Approach 1 and Approach 2: Approach 1 asks the model for a reason in words — useful for a person reading the logs later to understand *why* a message got routed a certain way. Approach 2 asks for a number instead — useful for setting an actual, tunable threshold (`if confidence < 60: decision = "unclear"`) that you can adjust without re-reading log text. A stricter production router would likely want both: a reason for humans, a number for automated decisions — but this project only needs one to prove the idea.

Difference from Intermediate: Intermediate's model-call router always forces a binary answer, even when the model itself isn't sure — it just silently picks the more likely of the two anyway. Advanced adds a genuine third outcome for that case, plus a record (a reason or a number) of how confident the router actually was, not just what it decided.

Which one should you use, and why? Build the keyword router first (Basic) to prove the wiring works, since it's free and instant. Move to the model-call version (Intermediate Approach 2) once you want it to handle wording you didn't anticipate. Add the "unclear" path (Advanced Approach 1) once you've actually seen the router get an ambiguous message wrong — that's when a forced binary choice stops being good enough. Reach for confidence scoring (Advanced Approach 2) once you want to *automate* what happens on uncertain messages (like a configurable threshold) rather than just read about it in logs after the fact — which is close to what Project 4's real Supervisor will need.